

GCP -- Automated Docker Image Build Pipeline

Cloud Build . Artifact Registry . Cloud Run . GKE

Every git push automatically builds, stores, and deploys your Docker image -- zero manual steps

Step-by-step setup . Dockerfile best practices . Full production pipeline . May 2025

Pipeline Overview

Every time you push code, GCP automatically builds the Docker image, stores it securely, and deploys it. The pipeline uses four GCP services working together.

Git push → Cloud Build → Artifact Registry → Cloud Run / GKE

Service	Role
Cloud Build	CI engine -- executes cloudbuild.yaml, runs tests, builds Docker image, pushes to registry
Artifact Registry	Managed Docker registry inside GCP -- stores images with vulnerability scanning and IAM access control
Cloud Build Triggers	Watch your repo and fire builds automatically on push, PR, or release tag
Cloud Run / GKE	Deployment target -- Cloud Build deploys the freshly built image directly

Cloud Build gives you 120 free build-minutes per day. A typical Spring Boot build takes 3-5 minutes, so you can run 20-30 builds/day for free.

1

One-time Setup

Enable APIs, create registry, grant Cloud Build IAM permissions

1.1 Enable required APIs

```
enable_apis.sh
gcloud services enable \
  cloudbuild.googleapis.com \
  artifactregistry.googleapis.com \
  run.googleapis.com \
  container.googleapis.com
```

1.2 Create an Artifact Registry repository

```
create_registry.sh
gcloud artifacts repositories create my-docker-repo \
  --repository-format=docker \
  --location=us-central1 \
  --description="Docker images for my app"

# Your image URL pattern:
# us-central1-docker.pkg.dev/PROJECT_ID/my-docker-repo/IMAGE:TAG
```

1.3 Grant Cloud Build the required IAM permissions

Cloud Build uses its own service account. Without these bindings, every automated build will fail with "permission denied."

```
iam_bindings.sh
PROJECT_ID=$(gcloud config get-value project)
PROJECT_NUMBER=$(gcloud projects describe $PROJECT_ID \
  --format='value(projectNumber)')
CB_SA="${PROJECT_NUMBER}@cloudbuild.gserviceaccount.com"

# Push images to Artifact Registry
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${CB_SA}" \
  --role="roles/artifactregistry.writer"

# Deploy to Cloud Run
gcloud projects add-iam-policy-binding $PROJECT_ID \
  --member="serviceAccount:${CB_SA}" \
  --role="roles/run.admin"

# Act as the compute service account
gcloud iam service-accounts add-iam-policy-binding \
  "${PROJECT_NUMBER}-compute@developer.gserviceaccount.com" \
  --member="serviceAccount:${CB_SA}" \
  --role="roles/iam.serviceAccountUser"
```

2

Dockerfile Best Practices

Layer caching is the key to fast automated builds

Put things that change rarely near the top (base image, dependency install) and things that change often near the bottom (your source code). Cloud Build reuses cached layers, cutting build time from 4 minutes to under 90 seconds on the second build.

2.1 Spring Boot -- multi-stage Dockerfile

```

Dockerfile.springboot
# Stage 1: build the JAR
FROM maven:3.9-eclipse-temurin-17-alpine AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -q # cached unless pom.xml changes
COPY src ./src
RUN mvn package -DskipTests -q

# Stage 2: minimal runtime image (no JDK in production)
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
USER appuser
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-XX:+UseContainerSupport", \
    "-XX:MaxRAMPercentage=75.0", "-jar", "app.jar"]

```

2.2 Angular -- multi-stage Dockerfile

```

Dockerfile.angular
# Stage 1: build Angular SPA
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --silent # faster than npm install
COPY . .
RUN npm run build -- --output-path=dist --configuration=production

# Stage 2: Nginx serves the static files
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/nginx.conf
EXPOSE 8080
CMD ["nginx", "-g", "daemon off;"]

```

2.3 .dockerignore -- always add this

```

.dockerignore
target/
node_modules/
.git/
*.log
.env
test-results/

```

3

cloudbuild.yaml

The build definition -- lives in the root of your repo

3.1 Built-in substitution variables

Variable	Value
\$PROJECT_ID	Your GCP project ID
\$SHORT_SHA	First 7 characters of the commit hash -- use as image tag
\$COMMIT_SHA	Full commit hash
\$BRANCH_NAME	Git branch name
\$TAG_NAME	Git tag (if trigger is tag-based)
\$BUILD_ID	Unique build ID for this run

3.2 Basic cloudbuild.yaml -- test, build, push, deploy

```

cloudbuild.yaml
# cloudbuild.yaml -- Spring Boot + Cloud Run
steps:
  # 1. Run tests -- fail fast
  - name: 'maven:3.9-eclipse-temurin-17'
    id: 'run-tests'
    entrypoint: mvn
    args: ['test', '-B', '-q', '--no-transfer-progress']

  # 2. Build image with layer caching
  - name: 'gcr.io/cloud-builders/docker'
    id: 'build-image'
    waitFor: ['run-tests']
    args:
      - build
      - --cache-from
      - 'us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:latest'
      - -t
      - 'us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:$SHORT_SHA'
      - -t
      - 'us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:latest'
      - .

  # 3. Push both tags to Artifact Registry
  - name: 'gcr.io/cloud-builders/docker'
    id: 'push-image'
    waitFor: ['build-image']
    args: ['push', '--all-tags',
      'us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api']

  # 4. Deploy to Cloud Run
  - name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
    id: 'deploy'
    waitFor: ['push-image']
    entrypoint: gcloud
    args:
      - run deploy my-api
      - --image=us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:$SHORT_SHA
      - --region=us-central1 --allow-unauthenticated
      - --min-instances=0 --max-instances=10

options:
  machineType: 'E2_HIGHCPU_8'
  logging: CLOUD_LOGGING_ONLY
timeout: '1200s'

```

Use \$SHORT_SHA as your image tag so every image is traceable to a specific commit. Never overwrite the same tag in production.

4

Cloud Build Triggers

Automatic builds on push, PR, or release tag

```
triggers.sh
# Push-to-main trigger (most common)
gcloud builds triggers create github \
  --name="build-on-push-main" \
  --repo-name="my-app" \
  --repo-owner="my-github-org" \
  --branch-pattern="^main$" \
  --build-config="cloudbuild.yaml"

# PR trigger -- test only, no deploy
gcloud builds triggers create github \
  --name="test-on-pr" \
  --repo-name="my-app" \
  --repo-owner="my-github-org" \
  --pull-request-pattern=".*" \
  --build-config="cloudbuild-test-only.yaml"

# Release tag trigger -- builds on tags like v1.2.3
gcloud builds triggers create github \
  --name="build-on-release-tag" \
  --repo-name="my-app" \
  --repo-owner="my-github-org" \
  --tag-pattern="^v[0-9]+\.[0-9]+\.[0-9]+$" \
  --build-config="cloudbuild-release.yaml"

# Monorepo -- only rebuild API when api source changes
gcloud builds triggers create github \
  --name="build-api-only" \
  --repo-name="my-monorepo" --repo-owner="my-org" \
  --branch-pattern="^main$" \
  --build-config="services/api/cloudbuild.yaml" \
  --included-files="services/api/*"

# Manage triggers
gcloud builds triggers list
gcloud builds triggers run build-on-push-main --branch=main
gcloud builds list --limit=10
```

Branch pattern reference

Pattern	Matches
<code>^main\$</code>	Only the main branch
<code>^main\$ ^release/.*</code>	main and any release/* branch
<code>^feature/.*</code>	Any feature branch
<code>.*</code>	Every branch (usually too broad for production)

5

Deployment Targets

Cloud Run, GKE, or any target reachable with gcloud

5.1 Cloud Run -- canary deploy (recommended pattern)

```

deploy_canary.yaml
# Deploy new revision without sending traffic to it
- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  id: 'deploy-canary'
  entrypoint: gcloud
  args:
    - run deploy my-api
    - --image=us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:$SHORT_SHA
    - --region=us-central1
    - --no-traffic          # 0% traffic to new revision
    - --tag=candidate-$SHORT_SHA

# Send 20% traffic to the new revision
- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  id: 'shift-canary-traffic'
  waitFor: ['deploy-canary']
  entrypoint: gcloud
  args:
    - run services update-traffic my-api
    - --region=us-central1
    - --to-tags=candidate-$SHORT_SHA=20

# After monitoring -- promote to 100%:
# gcloud run services update-traffic my-api --to-latest --region=us-central1

```

5.2 GKE -- rolling update

```

deploy_gke.yaml
- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  entrypoint: bash
  args:
    - '-c'
    - |
      gcloud container clusters get-credentials my-cluster \
        --region=us-central1
      kubectl set image deployment/my-api \
        api=us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:$SHORT_SHA
      kubectl rollout status deployment/my-api --timeout=5m

```

5.3 Inject secrets from Secret Manager

Never put passwords or API keys directly in cloudbuild.yaml. Always reference Secret Manager.

```

secrets.yaml
# cloudbuild.yaml -- reference Secret Manager secrets
availableSecrets:
  secretManager:
    - versionName: projects/$PROJECT_ID/secrets/db-password/versions/latest
      env: 'DB_PASSWORD'

steps:
- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  secretEnv: ['DB_PASSWORD']
  entrypoint: gcloud
  args:
    - run deploy my-api
    - --update-secrets=DB_PASSWORD=db-password:latest
    - --region=us-central1

```

6

Full Production Pipeline

Test → SonarQube → Build → Push → Scan → Staging → Integration tests → Prod canary

Stages 1-3: Test, build, push (cloudbuild-production.yaml)

```

# STAGE 1: TEST
- name: 'maven:3.9-eclipse-temurin-17'
  id: 'unit-tests'
  args: ['test', '-B', '-q', '--no-transfer-progress']

- name: 'maven:3.9-eclipse-temurin-17'
  id: 'sonar-scan'
  waitFor: ['unit-tests']
  args: ['sonar:sonar', '-Dsonar.login=${SONAR_TOKEN}']
  secretEnv: ['SONAR_TOKEN']

# STAGE 2: BUILD IMAGE (parallel cache pull)
- name: 'gcr.io/cloud-builders/docker'
  id: 'pull-cache'
  waitFor: ['-'] # runs immediately, parallel with tests
  args: ['pull', 'us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:latest']
  allowFailure: true # ok if no cache exists yet

- name: 'gcr.io/cloud-builders/docker'
  id: 'build-image'
  waitFor: ['unit-tests', 'pull-cache']
  args:
    - build
    - --cache-from
    - 'us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:latest'
    - -t
    - 'us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:${SHORT_SHA}'
    - -t
    - 'us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:latest'
    - .

# STAGE 3: PUSH AND VULNERABILITY SCAN
- name: 'gcr.io/cloud-builders/docker'
  id: 'push-image'
  waitFor: ['build-image']
  args: ['push', '--all-tags',
        'us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api']

- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  id: 'scan'
  waitFor: ['push-image']
  entrypoint: bash
  args: ['-c', 'gcloud artifacts docker images describe \
    us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:${SHORT_SHA} \
    --show-package-vulnerability 2>/dev/null || true']

```

Stages 4-6: Staging, integration tests, production canary

```

# STAGE 4: DEPLOY TO STAGING
- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  id: 'deploy-staging'
  waitFor: ['push-image']
  entrypoint: gcloud
  args:
    - run deploy staging-api
    - --image=us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:$SHORT_SHA
    - --region=us-central1 --allow-unauthenticated
    - --update-secrets=DB_PASSWORD=staging-db-password:latest

# STAGE 5: INTEGRATION TESTS AGAINST STAGING
- name: 'node:20-alpine'
  id: 'integration-tests'
  waitFor: ['deploy-staging']
  entrypoint: sh
  args:
    - '-c'
    - |
      npm ci --silent
      STAGING_URL=$(gcloud run services describe staging-api \
        --region=us-central1 --format='value(status.url)')
      API_BASE_URL=$STAGING_URL npm run test:integration

# STAGE 6: PRODUCTION CANARY DEPLOY
- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  id: 'deploy-prod'
  waitFor: ['integration-tests']
  entrypoint: gcloud
  args:
    - run deploy production-api
    - --image=us-central1-docker.pkg.dev/$PROJECT_ID/my-docker-repo/api:$SHORT_SHA
    - --region=us-central1 --no-traffic
    - --tag=candidate-$SHORT_SHA
    - --update-secrets=DB_PASSWORD=prod-db-password:latest
    - --min-instances=2 --max-instances=50

- name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
  id: 'canary-traffic'
  waitFor: ['deploy-prod']
  entrypoint: gcloud
  args:
    - run services update-traffic production-api
    - --region=us-central1
    - --to-tags=candidate-$SHORT_SHA=20

availableSecrets:
  secretManager:
    - versionName: projects/$PROJECT_ID/secrets/sonar-token/versions/latest
      env: 'SONAR_TOKEN'
options:
  machineType: 'E2_HIGHCPU_8'
  logging: CLOUD_LOGGING_ONLY
  env: ['DOCKER_BUILDKIT=1']
timeout: '1800s'

```

Monitoring and Troubleshooting

```

monitoring.sh
# View recent builds
gcloud builds list --limit=10

# Stream live logs for a running build
gcloud builds log BUILD_ID --stream

# Check images in Artifact Registry
gcloud artifacts docker images list \
  us-central1-docker.pkg.dev/PROJECT_ID/my-docker-repo

# Check vulnerability scan results
gcloud artifacts docker images describe \
  us-central1-docker.pkg.dev/PROJECT_ID/my-docker-repo/api:TAG \
  --show-package-vulnerability

# Check Cloud Run revisions after deploy
gcloud run revisions list --service=my-api --region=us-central1

# Promote canary to 100% after monitoring
gcloud run services update-traffic my-api \
  --region=us-central1 --to-latest

# Rollback to a previous revision
gcloud run services update-traffic my-api \
  --region=us-central1 \
  --to-revisions=PREVIOUS_REVISION_NAME=100

```

Quick Reference

What you want	Command
Submit a build manually	<code>gcloud builds submit --config=cloudbuild.yaml .</code>
List all triggers	<code>gcloud builds triggers list</code>
Run a trigger manually	<code>gcloud builds triggers run TRIGGER_NAME --branch=main</code>
View build history	<code>gcloud builds list --limit=20</code>
Stream build logs	<code>gcloud builds log BUILD_ID --stream</code>
List images in registry	<code>gcloud artifacts docker images list REPO_URL</code>
Scan image for CVEs	<code>gcloud artifacts docker images describe IMAGE_URL --show-package-vulnerability</code>
List Cloud Run revisions	<code>gcloud run revisions list --service=SERVICE --region=REGION</code>
Promote canary to 100%	<code>gcloud run services update-traffic SERVICE --to-latest --region=REGION</code>
Rollback to previous	<code>gcloud run services update-traffic SERVICE --to-revisions=NAME=100 --region=REGION</code>

